

CS571: Programming Languages

Assignment 4. Solution

Problem 1 [8pt] Assume that the 32-bits binary below is stored in the memory:

0100 0001 0111 1000 0000 0000 0000 0000

What will be the value of it if it is interpreted as

- a) (2pt) 32-bit integer?
- b) (2pt) 32-bit single-precision floating number?
- c) (2pt) two 16-bit short signed int?
- d) (2pt) 4 characters in ASCII code?

Solution:

- integer: 1098383360
- floating point number: $2^{130-127} \times 1.1111b = 1111.1b = 15.5$
- two 16-bit short signed int: 16760, 0
- characters: A, x, NULL, NULL

Problem 2 [10pt] Consider the following C declaration:

```
struct S1 {int a; double b; char c;};  
union S2 {float a; char b;};  
union U {  
    struct S1 s1;  
    union S2 s2;  
} u;  
union U arr[5][10];
```

Assume the machine has 1-byte characters, 4-byte integers, 4-byte floating numbers, and 8-byte double-precision floating numbers. Assume the compiler does not reorder the fields and the compiler might add padding in between fields and at the end of the struct to ensure that all fields start at a memory location with proper alignment. The size of the struct will also be a multiple of its largest alignment requirement. **You must justify your answer with explanation and/or memory layout diagrams or you will receive no credit for the assignment.**

- a) (2pt) What is the size of a struct S1 value?

Solution: 24 bytes.

a	a	a	a
padding	padding	padding	padding
b	b	b	b
b	b	b	b
c	padding	padding	padding
padding	padding	padding	padding

b) (2pt) What is the size of a union S2 value?

Solution: 4 bytes.

a/b	a	a	a
-----	---	---	---

c) (2pt) What is the size of a union U value u?

Solution: $\max(24, 4) = 24$ bytes.

s1/s2	s1/s2	s1/s2	s1/s2
s1	s1	s1	s1
s1	s1	s1	s1
s1	s1	s1	s1
s1	s1	s1	s1
s1	s1	s1	s1

d) (2pt) If the memory address of u starts from 3000, what is the start address of u.s1.b? What is the start address of u.s2.b?

Solution: The start address of u.s1.b is 3008, the start address of u.s2.b is 3000.

3000	u.s1.a/u.s2.a/u.s2.b	u.s1.a/u.s2.a	u.s1.a/u.s2.a	u.s1.a/u.s2.a
	padding	padding	padding	padding
3008	u.s1.b	u.s1.b	u.s1.b	u.s1.b
	u.s1.b	u.s1.b	u.s1.b	u.s1.b
	u.s1.c	padding	padding	padding
	padding	padding	padding	padding

e) (2pt) What is the size of the two dimensional array arr? If arr[0][0] is at address 3000, what is the address of arr[3][5]? And what is the address of arr[3][5].s1.b?

Solution: The size of the array is $24 \times 5 \times 10 = 1200$ bytes. The starting address of arr[3][5] is $3000 + 24 \times (3 \times 10 + 5) = 3840$. The address of arr[3][5].s1.b is $3840 + 8 = 3848$.

Each box below is 24 bytes.

3000	arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]	arr[0][4]	arr[0][5]	arr[0][6]	arr[0][7]	arr[0][8]	arr[0][9]
3240	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]	arr[1][4]	arr[1][5]	arr[1][6]	arr[1][7]	arr[1][8]	arr[1][9]
3480	arr[2][0]	arr[2][1]	arr[2][2]	arr[2][3]	arr[2][4]	arr[2][5]	arr[2][6]	arr[2][7]	arr[2][8]	arr[2][9]
3720	arr[3][0]	arr[3][1]	arr[3][2]	arr[3][3]	arr[3][4]	arr[3][5]	arr[3][6]	arr[3][7]	arr[3][8]	arr[3][9]
3960	arr[4][0]	arr[4][1]	arr[4][2]	arr[4][3]	arr[4][4]	arr[4][5]	arr[4][6]	arr[4][7]	arr[4][8]	arr[4][9]

Problem 3 [10pt] Consider the following C program:

```
int SumOfSquares(int n) {
    if (n<=0)
        return 0;
    else
        return n*n + SumOfSquares(n-1);
}
```

a) (6pt) Write down a tail recursive implementation of function SumOfSquares in C language. You can use helper function in your solution.

Solution:

```

int SumOfSquares(int n) {
    return SumOfSquaresHelp(0, n);
}
int SumOfSquaresHelp(int a, int n) {
    if (n<=0)
        return a;
    else
        return SumOfSquaresHelp(a+n*n, n-1);
}

```

- b) (4pt) An “optimizing” compiler will often be able to generate efficient code for recursive functions when they are tail-recursive. Refer to activation record, briefly explain how a compiler may “reuse” the same activation record for your solution in a).

Solution: Because in a tail-recursive function, the recursive call is the last thing in the function and the recursive call’s return value is directly used as the return value without further calculation, there is no need to use calling sequences to create a new stack frame (Activation Record) for the new recursive call to preserve current function call’s parameter and local variables for future calculation. The recursive call will reuse the current stack frame (Activation Record) by replacing parameter with new values, and return the answer directly from the new recursive call to the original caller.

Problem 4 [12pt] Consider the following C-like program. Write down what will be printed out when the parameters are passed (1) by value, (2) by reference, (3) by value return and (4) by name. For each answer, briefly explain how you derive it.

```

int x = 5, y = 6;
void foo(int a, int b) {
    x = a+b;
    b = a+a;
}
main () {
    foo(x, y);
    print(x, y);
}

```

Solution:

- (1) 11,6 (call by value)
- (2) 11,22 (call by reference)
- (3) 5,10 (call by value return)
- (4) 11, 22 (call by name)

Bonus

Problem 5 [2pt] This problem is from Exercise 9.4 of the Textbook. Consider the following (erroneous) program in C:

```

#include <stdio.h>
void foo() {
    int i;
    printf("%d ", i++);
}
int main() {
    int j;
    for (j=1; j<=10; j++) foo();
    return 0;
}

```

Local variable `i` in subroutine `foo` is never initialized. On many systems, however, the program will display repeatable behavior, printing 0 1 2 3 4 5 6 7 8 9. Suggest an explanation. Also explain why the behavior on other systems might be different, or non-deterministic.

Solution: In most language implementations, the activation records for different instances of `foo` will occupy the same space in the stack. Local variable `i` is never initialized (that's why the program is erroneous), but if the stack space has not been used for anything else in the meantime it may "inherit" a value from the previous instance of the routine. If the stack is created from space filled by zeros by the operating system, and if the space occupied by `foo`'s activation record is not used for anything else before the first time `foo` is called, then `i` will start with the value zero in the first iteration of the loop.

On other systems, the stack might be created from space that is not filled with zeros and therefore the first iteration of the loop `i` might not start with the value zero.

Problem 6 [2pt] RSA is an algorithm used by modern computers to encrypt and decrypt messages. The core of RSA is the computation of $(p^k \bmod n)$ where k is confidential (i.e., with k , an attacker can derive the private key). For efficiency, some implementations of RSA use a *window-based* modular exponentiation, which creates a two-dimensional array, say A , so that $A[i][j]$ stores the j -th byte of $(p^i \bmod n)$, where $0 \leq i \leq 2^3 - 1$, assuming a window size of 3. Note that 384 bytes are typically needed for each $A[i]$ since p, k, n are large numbers. The algorithm works as follows:

```

for (i=1; i<key_length; i=i+3):
    v = value of k's bits from i to (i+2) // value of the current window
    some computation that reads A[v][0] to A[v][383]

```

Security experts found that for an attacker who can observe the sequence of accessed memory addresses *except for the last 6 bits* (essentially, this is an attacker who can observe which 64-byte cache line is accessed but not the offset in the cache line), the entire key might be revealed if the implementation is written in C. But the same implementation in some other language is immune to such attack. Use the array memory layout covered in class to explain briefly how this could happen (assume A is aligned with cache line, i.e., A start from an address where all last 6 bits are zeros).

Solution: Languages with column-major layout for multiple dimensional arrays are not vulnerable, since as long as A is aligned, reading any byte among $A[0][j], A[1][j], \dots, A[7][j]$ will only exhibit difference in the last 3 bits, which is not visible to the attacker. Hence, the attacker cannot observe which v is accessed in each loop iteration. On the other hand, a row-major layout will reveal the value of v , since $A[0][j], A[1][j], \dots, A[7][j]$ all map to different cache lines in this memory layout. Hence, by observing which cache line is accessed, an attacker can reveal the value of v , hence the value of k .